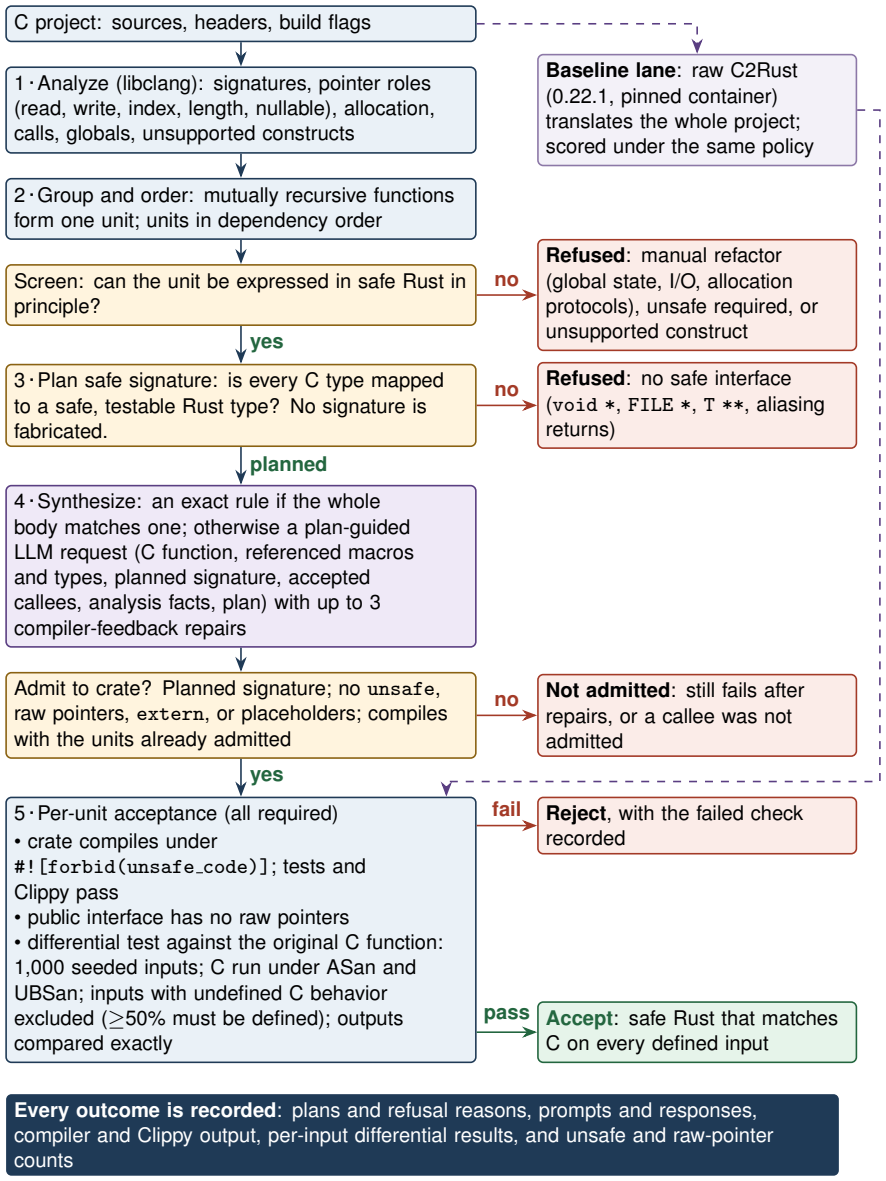
C project: sources, headers, build flags
1 · Analyze (libclang): signatures, pointer roles (read, write, index, length, nullable), allocation, calls, globals, unsupported constructs
2 · Group and order: mutually recursive functions form one unit; units in dependency order
Screen: can the unit be expressed in safe Rust in principle?
no
Refused: manual refactor (global state, I/O, allocation protocols), unsafe required, or unsupported construct
yes
3 · Plan safe signature: is every C type mapped to a safe, testable Rust type? No signature is fabricated.
no
Refused: no safe interface (void *, FILE *, T **, aliasing returns)
planned
4 · Synthesize: an exact rule if the whole body matches one; otherwise a plan-guided LLM request (C function, referenced macros and types, planned signature, accepted callees, analysis facts, plan) with up to 3 compiler-feedback repairs
Admit to crate? Planned signature; no unsafe, raw pointers, extern, or placeholders; compiles with the units already admitted
no
Not admitted: still fails after repairs, or a callee was not admitted
yes
5 · Per-unit acceptance (all required)
• crate compiles under #![forbid(unsafe_code)]; tests and Clippy pass
• public interface has no raw pointers
• differential test against the original C function: 1,000 seeded inputs; C run under ASan and UBSan; inputs with undefined C behavior excluded (≥50% must be defined); outputs compared exactly
fail
Reject, with the failed check recorded
pass
Accept: safe Rust that matches C on every defined input
Baseline lane: raw C2Rust (0.22.1, pinned container) translates the whole project; scored under the same policy
Every outcome is recorded: plans and refusal reasons, prompts and responses, compiler and Clippy output, per-input differential results, and unsafe and raw-pointer counts